

PCOS Research System

From API Ingestion to Autonomous Scientific Discovery

This document is a **technical reference** for the PCOS computational research system built at Yale. It explains, in order, every component of the system: what it does, how it works, and why it was designed that way. It also explains how to build a **Claude Code Skill** from scratch.

Topics covered: Data ingestion from scientific APIs · ETL pipeline · Vector embeddings · LLM classification and extraction · Entity normalization (4-layer architecture) · Anomaly detection · Autonomous research agent (ReAct loop, 6 tools) · Building a Claude Code Skill.

Yale University · Computational Biomedical AI · April 2026

0 1

1	Data Ingestion — Fetching Scientific Literature	3
1.1	The Three Data Sources	3
1.2	What is a REST API?	3
1.3	Rate Limiting and Idempotency	3
2	The Knowledge Base — ETL, SQLite, and ChromaDB	3
2.1	ETL: Extract, Transform, Load	4
2.2	SQLite — Structured Data	4
2.3	ChromaDB — Semantic Search	4
3	AI Processing — Labeling, Extraction, and Schema v1.1	4
3.1	Step 1: Classification (Labeling)	5
3.2	Step 2: Structured Extraction (Schema v1.1)	5
4	Entity Normalization — 4-Layer Architecture	5
4.1	The Problem and Solution	6
4.2	The 4-Layer Pipeline	6
4.3	The Alias Learner	6
4.4	Why Not Use a Standard Medical Ontology?	6
5	Anomaly Detection — Signals Without Prior Hypotheses	7
5.1	Three Types of Signal	7
5.2	The Historical Precedent: Metformin	7
6	The Research Agent — Autonomous Investigation	7
6.1	What is an AI Agent?	7
6.2	The Six Tools	8
6.3	Tool Use: How the LLM Calls Functions	8
6.4	Why temperature = 0.15?	8
6.5	The Output	8
7	Building a Claude Code Skill	8
7.1	The File Structure	9
7.2	The pcos-agent Skill: Four Modes	9
7.3	Design Principles of a Good Skill	9

1 1

The system starts by pulling raw data from three external scientific databases. Each one contributes something different, and together they form the knowledge base the AI will learn from.

The Three Data Sources

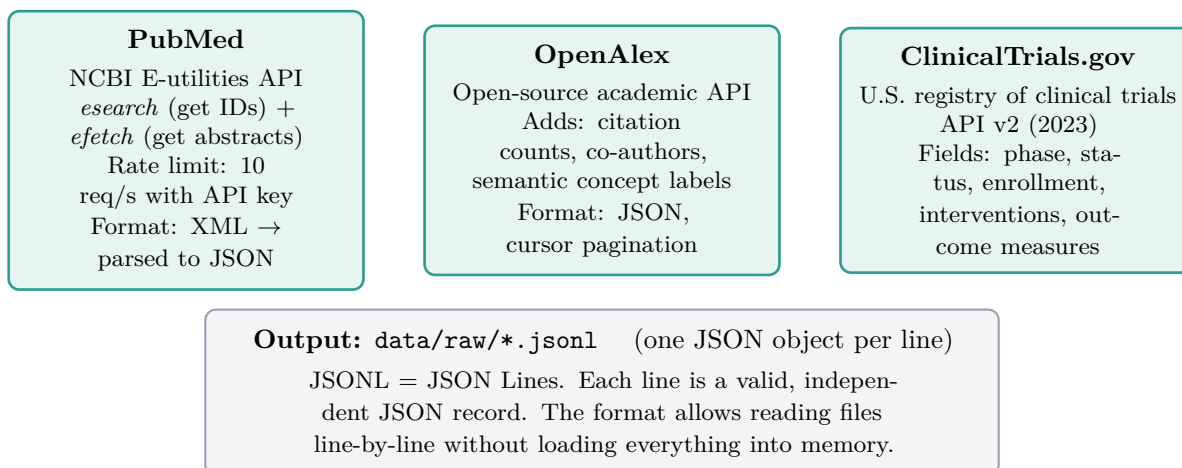


Figure 1: The three data sources and their output format

What is a REST API?

A **REST API** (Representational State Transfer Application Programming Interface) is a standard way to communicate with a remote server over the internet. Your Python script sends an **HTTP GET request** with parameters, and the server responds with structured data (usually JSON or XML).

Example: PubMed ESearch call

```
GET https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi
?db=pubmed&term=polycystic+ovary+syndrome[MeSH]&retmax=10000
```

Response: a list of PMIDs (PubMed IDs). Then a second call to EFetch retrieves the full abstract, authors, journal, and year for each ID.

Rate Limiting and Idempotency

Rate limiting is a server-side restriction on how many requests you can make per second. PubMed allows 10 requests/second with an API key; without one, only 3/second. The scripts use `time.sleep(0.1)` between requests to stay within limits.

Idempotency means running a script multiple times produces the same result as running it once. All ingestion scripts use **INSERT OR IGNORE** in SQLite — if an article already exists, it is silently skipped. This makes reruns safe.

2 1

Raw JSONL files are useful for storage, but not for querying. The ETL step normalizes and loads everything into two complementary databases.

ETL: Extract, Transform, Load

ETL is the foundational pattern of data engineering. Every data pipeline that moves information from one system to another follows this structure:

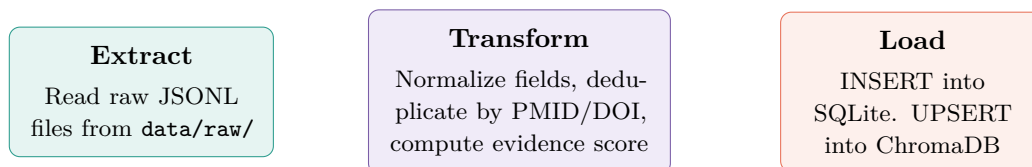


Figure 2: The ETL pattern applied in `build_knowledge_base.py`

SQLite — Structured Data

SQLite is a relational database stored in a single file (`pcos_research.db`). It supports full SQL and is ideal for projects where a single process writes data. Its critical constraint: **only one writer at a time**. This is why the entire pipeline must run sequentially, never in parallel.

Two important SQLite settings used throughout the project:

- `PRAGMA journal_mode=WAL` — Write-Ahead Logging mode, which allows readers (SELECT queries) to run simultaneously with a writer (INSERT), preventing read locks.
- `connection timeout=60` — If the database is locked by another process, wait up to 60 seconds before raising an error, rather than failing immediately.

The database has seven tables: `articles`, `article_llm_labels`, `article_extractions`, `entity`, `article_entity_links`, `intervention_signals`, and `clinical_trials`. The relationships between them form a knowledge graph of interventions, populations, and outcomes.

ChromaDB — Semantic Search

ChromaDB is a **vector database**: instead of storing text, it stores numerical **embeddings** — lists of 768 numbers that encode the meaning of a text. When you search ChromaDB, you do not search for exact words; you search for *meaning*, and it returns the most semantically similar documents.

What is an Embedding?

An embedding is a representation of text as a point in high-dimensional space. Two texts about similar topics will have points that are close together. This is computed by a model called **nomic-embed-text**, which runs locally via Ollama. The 768-dimensional space means each text becomes a list of 768 numbers, where numbers at similar positions encode similar semantic features.

The similarity between two embeddings is measured with **cosine similarity**: a value between 0 (unrelated) and 1 (identical meaning). ChromaDB's internal index uses the **HNSW algorithm** (Hierarchical Navigable Small World) to find the nearest neighbors in milliseconds, even across 12,155 vectors.

3 1

The pipeline has two LLM-powered steps that convert free-form abstracts into structured scientific data.

Step 1: Classification (Labeling)

The classifier reads each article’s title and abstract and assigns three fields. This is a **few-shot prompting** task: the prompt includes 4–5 labeled examples so Gemma understands the expected format and standard.

Field	Values	Meaning
study_type_llm	RCT, meta-analysis, cohort, case study, narrative...	What kind of study is this?
evidence_tier_llm	tier_1 to tier_4	Quality level (1 = gold standard, 4 = opinion)
utility_score_llm	0.0 – 1.0	How relevant is this article to PCOS research?

Step 2: Structured Extraction (Schema v1.1)

The extractor is more demanding: it reads each abstract and fills out a structured record with specific scientific fields. Version 1.1 adds five new **population subgroup fields**, which are critical for detecting subpopulation variance.

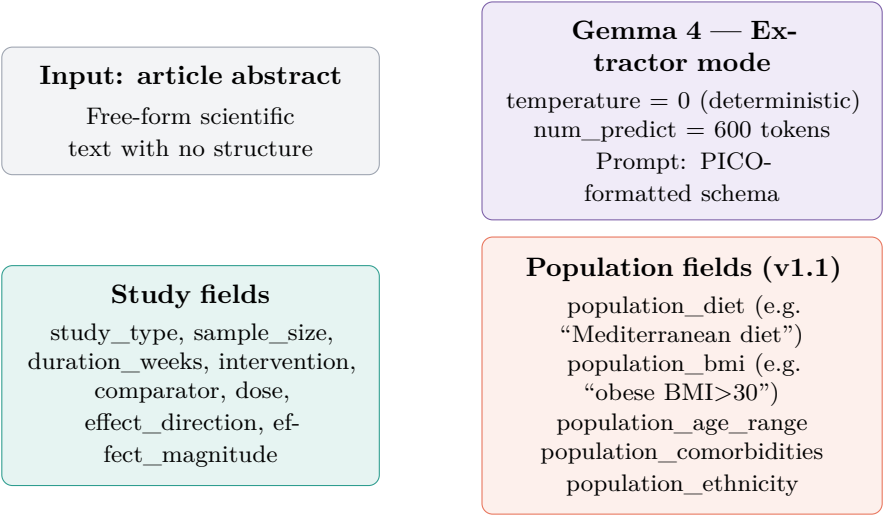


Figure 3: Extraction schema v1.1 — study fields and new population fields

Why do population fields matter? Without them, you can only say “berberine works in 8 of 10 studies.” With them, the anomaly scanner can discover that berberine works in 6/6 studies with obese patients, but only 2/4 with normal-weight patients — a subpopulation signal invisible without this data.

Prioritization: the extractor does not process all 12,155 articles at once. It selects those with evidence_tier_llm = 'tier_1' and high utility_score_llm first, maximizing the scientific return per LLM call.

4 1

Drug names in scientific papers are inconsistent. The same compound appears as “metformin”, “Metformin HCl”, “metformin 1500 mg/day”, and “glucophage”. Without normalization, these are treated as different entities and their statistics cannot be aggregated. The entity normalizer solves this.

The Problem and Solution

Canonical Entity

A **canonical entity** is the single, official name chosen to represent all variant spellings. The normalizer maps every raw term found in a paper back to its canonical name. The system maintains a **SEED dictionary** — a curated Python dict of 76+ canonical entities with their known aliases.

The 4-Layer Pipeline

Each raw term passes through layers in order, stopping at the first successful match.

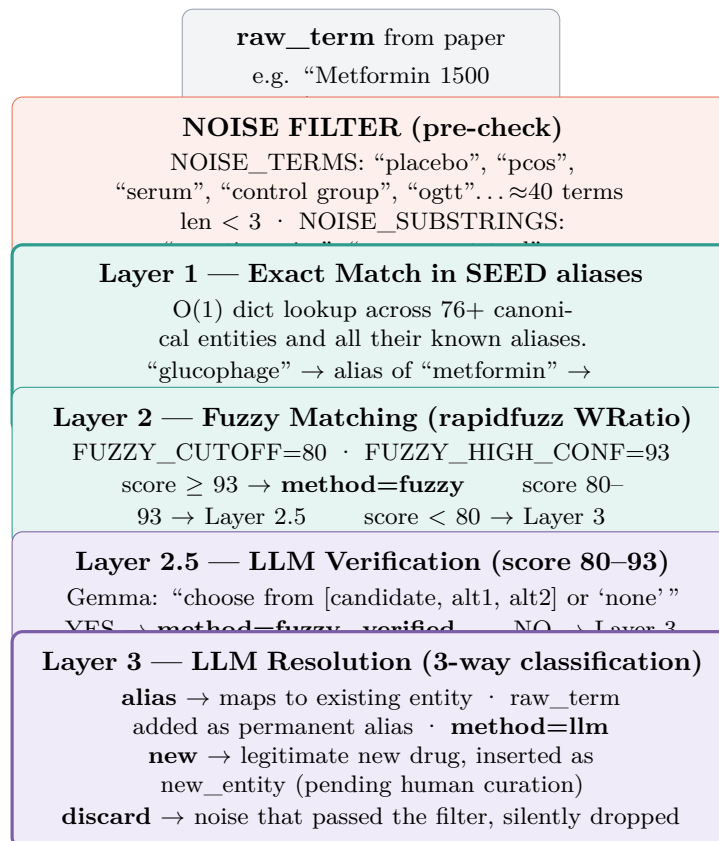


Figure 4: Entity Normalizer v4 — 4-layer resolution pipeline

The Alias Learner

One design feature worth highlighting: when Layer 3 classifies a term as “alias”, it does two things simultaneously. It creates the entity link *and* permanently adds the raw term to the SEED aliases. The next time that term appears in any paper, it is resolved instantly in Layer 1 — no LLM call needed. **The system learns from every run**, progressively reducing the number of LLM calls required.

Why Not Use a Standard Medical Ontology?

Medical ontologies like MeSH or ChEBI contain hundreds of thousands of terms. Querying them for every unknown term would be slow and would return irrelevant matches. The curated SEED with 76+ targeted entities covers over 95% of interventions that actually appear in PCOS literature — a much better precision/recall tradeoff.

51

Most scientific research starts with a hypothesis and tests it. This system does the opposite: it scans the aggregated data for patterns that *nobody was looking for*. This is possible because the entity normalizer connects hundreds of papers through their canonical entities.

Three Types of Signal

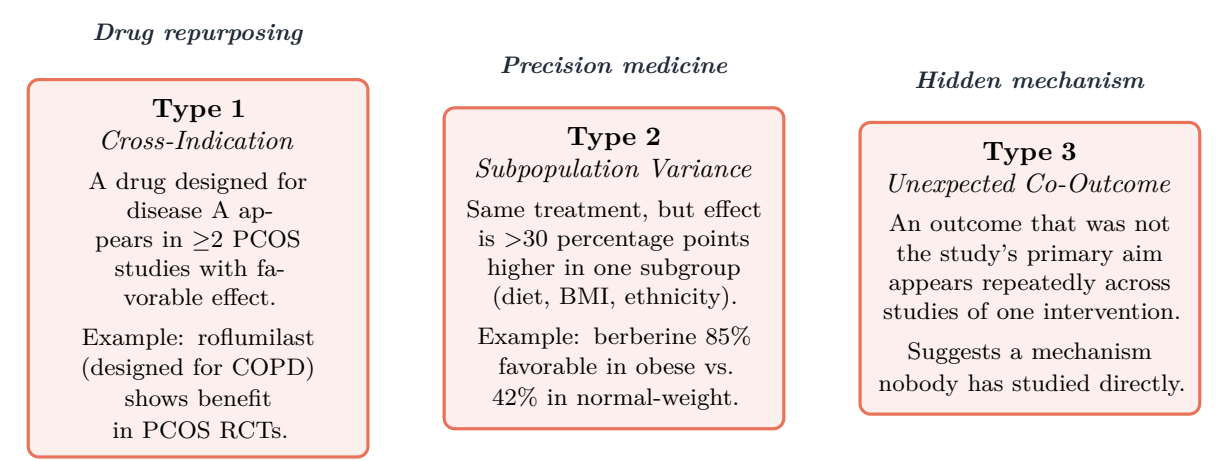


Figure 5: Three types of emergent signal detected by `anomaly_scan_v2.py`

The Historical Precedent: Metformin

Metformin was developed in the 1950s for type 2 diabetes. Its use in PCOS emerged in the 1990s when clinicians noticed that insulin-resistant PCOS patients responded to it. Today it is the first-line PCOS treatment. The anomaly scanner is designed to find the *next metformin*: a drug from another field quietly showing benefit in PCOS data, before any research team explicitly studies it.

61

When the anomaly scanner finds a signal, a human investigator would normally spend days reading the relevant literature. The research agent does this autonomously in 5–15 minutes, using a pattern called the **ReAct loop**.

What is an AI Agent?

A standard LLM answers a question in a single step. An **AI agent** can take multiple steps, using **tools** between them. Each step: reason about what information is needed, call a tool to get it, observe the result, reason again. This loop repeats until the agent has enough information to draw a conclusion.

ReAct: Reason + Act

ReAct (Yao et al., 2022) is the foundational pattern for agentic AI. **Reason:** the LLM explains what it knows and what it needs. **Act:** the LLM invokes a tool (search, SQL, web). **Observe:** the tool result is added to the context. The cycle repeats. The LLM always sees the full history of what it has found so far.

The Six Tools

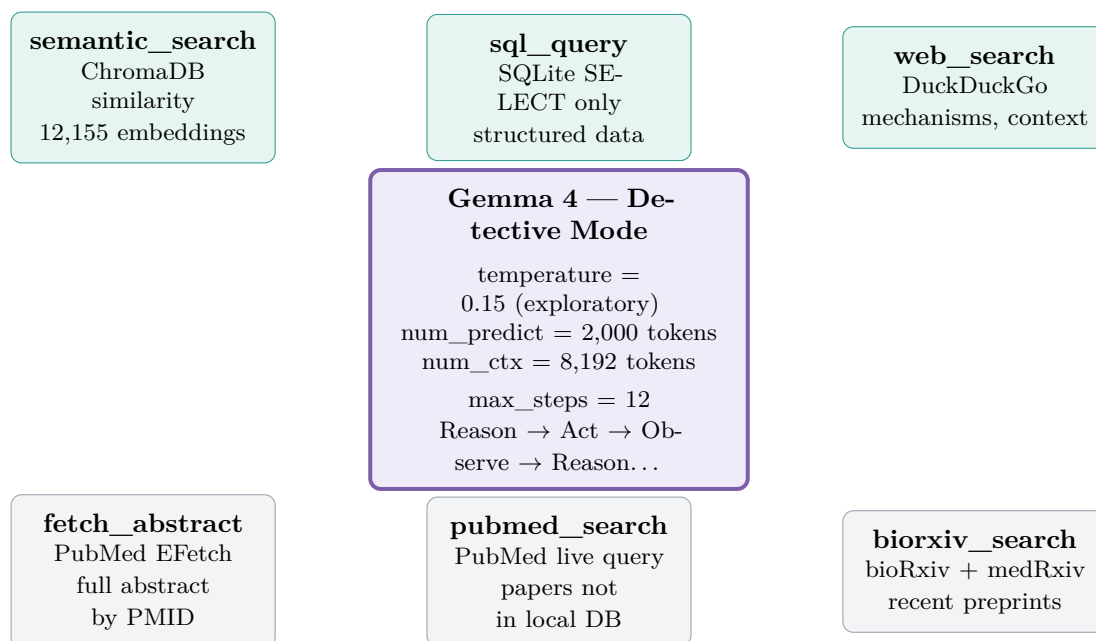


Figure 6: The research agent and its 6 tools. The agent calls each tool and receives its result back into context.

Tool Use: How the LLM Calls Functions

Tool use (also called *function calling*) is a capability of modern LLMs: instead of generating only text, the model can output a structured JSON object requesting a specific function call. The host code (Python) executes the function and returns the result as the next message. Ollama supports this natively via the `tools` parameter in `/api/chat`.

Why temperature = 0.15?

For deterministic tasks (extraction, classification), temperature is set to 0 — the model always produces the same output for the same input, which is necessary for reproducibility. The research agent uses 0.15 instead: a small amount of randomness allows it to explore different investigative paths when the same signal is investigated multiple times, rather than always following the identical reasoning chain.

The Output

After up to 12 steps, the agent produces: a natural-language reasoning trace (the full investigation), a structured JSON conclusion with a `mechanistic_hypothesis`, a `plausibility_score` (0.0–1.0), a list of `research_gaps`, and a `proposed_experiment`. The full report is saved to `reports/research_agent_<name>.md`.

A **Claude Code Skill** is a plain-text file that gives Claude a specialized behavioral mode when a specific command is invoked. It is not code — it is *instructions* written in markdown that Claude reads and follows.

The File Structure

Every skill is a single file: `~/.claude/skills/<skill-name>/SKILL.md`.

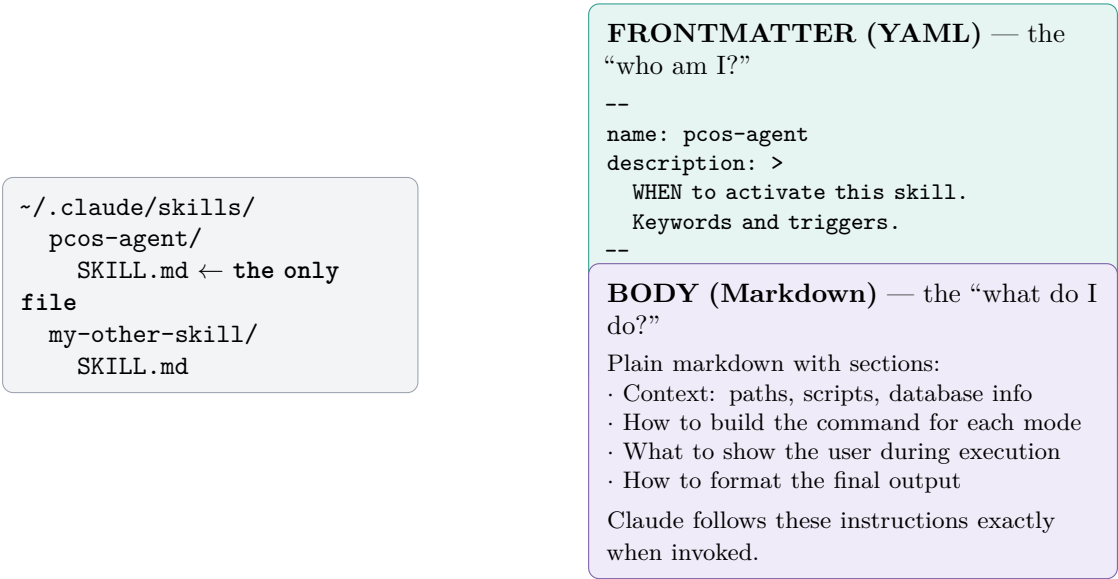


Figure 7: Anatomy of a SKILL.md file: frontmatter (when to activate) and body (what to do)

The pcos-agent Skill: Four Modes

The `pcos-agent` skill wraps the entire research system into four commands.

Command	What it does	Runs
<code>/pcos-agent investigate entity</code>	Launches Gemma 4 as a detective scientist. Runs the full ReAct loop with 6 tools to investigate a drug or supplement in PCOS context. Produces a mechanistic hypothesis.	<code>research_agent.py -entity</code>
<code>/pcos-agent scan</code>	Runs the anomaly scanner on the full database. Detects cross-indication signals, subpopulation variance, and unexpected co-outcomes. Shows results in a formatted table.	<code>anomaly_scan_v2.py</code>
<code>/pcos-agent status</code>	Queries the database and shows key statistics: article counts by tier, top interventions by study count, and a list of recent research reports.	Python inline SQL
<code>/pcos-agent review name</code>	Reads an existing research report from <code>reports/</code> and presents the key findings: plausibility, hypothesis, evidence, gaps.	Read tool on <code>.md</code> file

Design Principles of a Good Skill

A skill is only as useful as how completely it briefs Claude. The body of `SKILL.md` must be **self-contained**: Claude cannot “look around” in your filesystem to find paths, scripts, or data locations — every piece of information it needs must be written explicitly in the skill.

Bad skill body: “Execute the research script.”

Good skill body: “Run `.venv/Scripts/python.exe scripts/research_agent.py -entity [X] -max-steps 12` from the directory `C:\Users\emaza\OneDrive\Escritorio\CODEX\PRUEBA1`. The venv is at `.venv/Scripts/python.exe` (not system Python). The output is printed to stdout and saved to `reports/research_agent_<name>.md`.”